

Computer Architecture Interview Guide

1. What is Computer Architecture?

Computer Architecture is how a computer is designed internally so it can execute programs efficiently.

It includes:

- CPU
- Memory
- Cache
- Buses
- Registers
- Input/Output devices

Think of it as the blueprint of a computer.

2. Basic CPU Execution

Suppose we write:

```
int c = a + b;
```

The CPU doesn't understand C++. The compiler converts it into machine instructions.

Then the CPU performs:

Read a ↓

Read b ↓

Add ↓

Store result

Every program eventually becomes these tiny operations.

3. CPU Pipeline ★ ★ ★ ★ ★

One of the most common interview topics.

Instead of doing one instruction at a time, the CPU overlaps multiple instructions.

Imagine a car factory. Instead of building one complete car...

Station 1: Install Engine → Station 2: Install Doors → Station 3: Paint → Station 4: Testing

Many cars move simultaneously. CPU does the same.

Pipeline stages:

Instruction → Fetch → Decode → Execute → Memory → Write Back

Pipeline Stages Details

- **Fetch:** CPU reads instruction from memory. (Example: ADD R1, R2)
- **Decode:** CPU understands what instruction means. (Example: "Oh... This is ADD.")
- **Execute:** ALU performs calculation. (Example: $5 + 10 = 15$)
- **Memory:** If instruction needs RAM, CPU reads or writes memory.
- **Write Back:** Result stored into register. Done.

Interview Question

Why do CPUs use pipelining?

Answer:

It improves performance by executing multiple instructions simultaneously at different stages rather than waiting for one instruction to complete before starting the next.

4. Pipeline Hazards (Surface Level)

Sometimes the pipeline gets stuck. There are three types of hazards:

Data Hazard

Instruction 2 needs result from Instruction 1.

Example:

ADD R1, R2

SUB R3, R1

Second instruction cannot start until first finishes.

Solution: Forwarding or pipeline stall.

Control Hazard

Occurs after if(), for(), while(), or a branch. CPU doesn't know which instruction comes next.

Solution: Branch Prediction.

Structural Hazard

Two instructions need same hardware. Example: Two instructions want memory at same time. One must wait.

5. Registers ★ ★ ★ ★ ★

Registers are tiny memories inside CPU. They are very fast.

RAM → Register → ALU

CPU first copies data from RAM into registers, then performs calculations.

Registers are much faster than RAM.

Interview Question

Why not store everything in registers?

Answer:

Registers are extremely fast but very small and expensive.

6. Memory Hierarchy ★ ★ ★ ★ ★

Speed decreases as we go down. Size increases as we go down.

Registers

↓

L1 Cache

↓

L2 Cache

↓

L3 Cache

↓

RAM

↓

SSD/HDD

Higher = Faster | Lower = Bigger

7. Cache Memory ★ ★ ★ ★ ★

CPU is extremely fast, but RAM is much slower. So CPU keeps recently used data in cache.

Example:

Suppose program repeatedly accesses "temperature". Instead of fetching from RAM every time... CPU stores it in cache. Next access is much faster.

Three cache levels:

- **L1:** Fastest, smallest, inside each core.

- **L2:** Larger, slightly slower.
- **L3:** Shared by all CPU cores. Largest cache.

Interview Question

Why is cache needed?

Answer:

Cache stores frequently accessed data close to the CPU, reducing memory access time and improving performance.

Cache Terminology:

- **Cache Hit:** Requested data found in cache. Very fast.
- **Cache Miss:** Data not in cache. CPU goes to RAM. Slower.

8. RAM vs ROM

RAM	ROM
Temporary Read + Write Loses data after power off	Permanent Stores firmware Doesn't lose data
Examples: Variables, Program execution	Examples: BIOS, Microcontroller firmware

9. SRAM vs DRAM

Very common interview topic.

SRAM (Static RAM)	DRAM (Dynamic RAM)
Fast Expensive Uses flip-flops No refresh needed Used for Cache	Cheaper Slower Needs refresh Used as Main Memory (RAM)

Interview Answer

Cache uses SRAM because it's faster. RAM uses DRAM because it's cheaper and can store much more data.

10. Stack vs Heap ★ ★ ★ ★

Stack	Heap
Function variables (e.g., int x; int y;) Automatic memory management Small Very fast	Dynamic memory (e.g., new, malloc()) Programmer manages it (manual) Large Slower

11. Virtual Memory

Programs think they have huge memory. Actually...

Program → Virtual Address → MMU → Physical RAM

OS converts virtual address into physical address.

Benefit: Programs don't interfere with each other (isolation/security).

12. Multi-Core Processor

Old CPU: 1 Core

Modern CPU: Core 1, Core 2, Core 3, Core 4

Multiple tasks run simultaneously.

Example:

Core 1: Browser

Core 2: Music

Core 3: Video

Core 4: Background apps

13. Bus ★ ★ ★ ★

A bus is simply a communication pathway. There are three important buses:

- **Data Bus:** Transfers actual data (e.g., 10101010).
- **Address Bus:** Tells memory "Where should I read/write?" (Example: Address 2000).
- **Control Bus:** Carries control signals (Examples: Read, Write, Interrupt, Clock).

14. Microarchitecture (Just One Line)

Architecture defines what a processor should do (instruction set and behavior).

Microarchitecture defines how it is actually implemented internally (pipeline, caches, execution units, branch predictor, etc.).

Example: Intel and AMD processors can both support the x86 instruction set (same architecture) but have different microarchitectures.

Questions They May Ask

Q: What is cache?

A: "A small, high-speed memory between the CPU and RAM that stores frequently accessed data."

Q: Why is cache faster than RAM?

A: "It is built using SRAM and is physically much closer to the CPU."

Q: Difference between SRAM and DRAM?

A:

- SRAM: Faster, expensive, used for cache.
- DRAM: Slower, cheaper, used for main memory.

Q: What is pipelining?

A: "Executing multiple instructions simultaneously at different stages to improve CPU throughput."

Q: Why are registers needed?

A: "They store data that the CPU is currently processing because accessing registers is much faster than accessing RAM."

Q: What is a bus?

A: "A communication channel that transfers data, addresses, and control signals between computer components."